# NoSQL injections

# A login without the password

`"password": "wrong"` → `{ "$ne": "" }`

A controlled comparison of five login configurations

Ercong Kee / Pheobe Neo / Kendrick Poon / Liu Qingyu

Ron et al., 2016 / controlled local reproduction

# A query is a test on a record

### DOCUMENT = A STORED RECORD

```
{ username: "alice", password: "…" }
```

Alice’s stored password is a non-empty string in the vulnerable lab.

---

### EQUALITY CONDITION

```
password: "wrong"
```

Does the password equal “wrong”?

### OPERATOR CONDITION

```
password: { $ne: "" }
```

Is the password different from an empty string?

**Predicate = condition** **$ne = not equal** **"" = empty string**

NoSQL models: document, key-value, graph, wide-column. Here: MongoDB documents.

MongoDB 7.0 $ne documentation / illustrative teaching record

# Where the login goes wrong

**THE ATTACKER CONTROLS THE REQUEST**




| Sends JSON | Builds query | Returns Alice | Remembers Alice | Allows access |
|---|---|---|---|---|

**The failure occurs when input changes the query condition.**

Known username, no direct database access, no changes to stored records

# The paper covers several attack boundaries

| INPUT ENTERS… | | WHAT CAN CHANGE |
|---|---|---|
| **Query conditions** | → | Operator object changes a condition<br>Tautologies / our experiment |
| **Query text** | → | Added logic changes which records match<br>“Union queries” / literature |
| **JavaScript source** | → | String becomes executable source<br>JavaScript injection / comparison |
| **A protocol command** | → | A separator introduces another command<br>Piggybacked queries / literature |
| **A trusted browser** | → | Browser sends an unwanted action<br>Origin violation / CSRF / adjacent issue |

**Which controls stop the highlighted operator route?**

Ron et al., 2016 / author manuscript and republication

# The password value becomes a condition

### 1 CLIENT REQUEST

```
{"username":"alice","password":{"$ne":""}}
```

### 2 APPLICATION COPIES THE VALUES

```
users.findOne({
  username: body.username,
  password: body.password
})
```

### 3 BOTH CONDITIONS MATCH

**username equals Alice**
**AND**
**password is not ""**

**A match becomes a session without password proof.**

For this record, the stored password is non-empty. $ne: "" also matches missing fields.

# An object and a string behave differently

**OBJECT**

```
{ "$ne": "" }
```



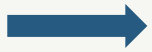
**MongoDB operator**

**CONDITION TRUE**
**Database returns Alice**

---

**STRING**

```
"{\"$ne\":\"\"}"
```



**Literal text**

**CONDITION FALSE**
**Login rejected**

**The parsed input type changes what the query means.**

# How we tested the controls

## 15 cases

Correct / wrong passwords
Operator objects
Unexpected types and fields

**5 CONFIGURATIONS**

Baseline

JavaScript disabled

Read-only DB account

String validation

Full hardened login

**2 RUNS**

Login decision
Protected access
Query count

## 150 recorded case results

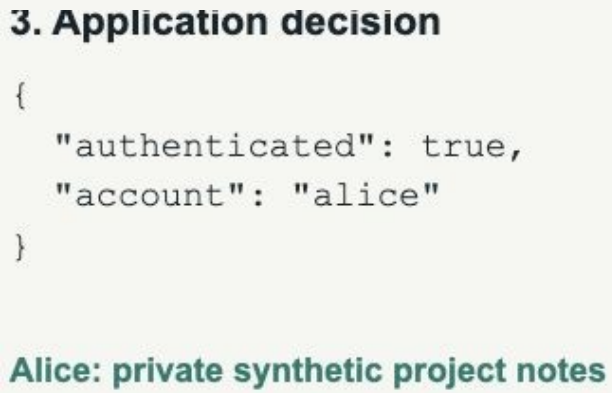
Control probes verify settings.
Repeated runs show local repeatability.

15 cases × 5 configurations × 2 runs / results.json

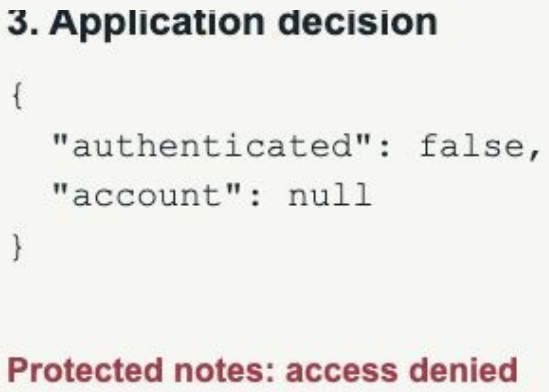
# Same attack, different login decision

| 01 | 02 | 03 | 04 |
|---|---|---|---|
| **Wrong password** | **Attack payload** | **Same attack after fix** | **Correct password** |
| **Login rejected** | **Attack succeeds**<br>**Logged in as Alice** | **Attack stopped**<br>**No database query** | **Legitimate login works** |

**CAPTURED ATTACK SUCCESS**




**CAPTURED ATTACK REJECTION**




Live lab: http://127.0.0.1:4400 Offline video: 90 seconds

Actual browser captures / local synthetic lab / 1 November 2026

# Which controls stopped the attack?

| Configuration | Attack outcome | Database query | Correct login |
|---|---|---|---|
| Baseline | **Attack succeeds (200)** | 1 query sent | Login works (200) |
| JavaScript disabled | **Attack succeeds (200)** | 1 query sent | Login works (200) |
| Read-only DB account | **Attack succeeds (200)** | 1 query sent | Login works (200) |
| String validation | **Attack stopped (400)** | No query sent | Login works (200) |
| Full hardened login | **Attack stopped (400)** | No query sent | Login works (200) |

Disabling scripting does not affect this query operator.
Read-only access still permits the one read needed by the attack.

Focal attack C04 / complete matrix in slides 17–18

# A safe login keeps the password out of the query

WHY THE ATTACK FAILS

**The password object is not text, so the app rejects it before MongoDB sees it.**

SAFE LOGIN FLOW

**01**

**Accept text only**

Username and password must be non-empty text

**02**

**Find the account**

MongoDB uses the username only

**03**

**Check the password**

App compares the input with the stored hash

**04**

**Create a session**

Only if the password matches

Local source / OWASP NoSQL and password-storage guidance

# Different injection types need different defences

**MEASURED IN OUR LAB**

**Object contains $ne** → **MongoDB reads a query condition** → **Changes which records match**

Control: for login fields, accept text only and build the query in application code.

---

**LITERATURE COMPARISON**

**Text inserted into JavaScript source** → **JavaScript engine executes the source** → **Changes program logic**

Control: do not build code from user text; disable unused scripting.

We measured the first row. The fixed $where probe checked the server setting only.

Literature comparison / MongoDB 7.0 $where documentation

# What our experiment shows

**1** **The attack succeeds**

Operator object logs the attacker in as Alice

**Session created**
**HTTP 200**

**2** **The attack is stopped**

Object rejected before MongoDB receives a query

**No query sent**
**HTTP 400**

**3** **Legitimate login still works**

Correct credentials retain protected access

**Access granted**
**HTTP 200**

Scope: one recorded stack, synthetic accounts and 15 request cases.

Recorded local experiment / results.json and results.csv

# The application must keep control of the query

## Choose the query structure.
## Validate values. Verify the password.

NEXT QUESTION

**How can flexible search preserve customer separation?**

**User-selected filter** + **Mandatory tenant rule**

Proposed test: allowed filters vs a builder that always adds the tenant rule.

# Recorded environment

| Component | Recorded value |
| --- | --- |
| MongoDB | 7.0.0 |
| Node.js | v23.6.1 |
| Express / native driver | 5.1.0 / 6.20.0 |
| Transport / parser | Loopback HTTP / JSON / express.json |
| Password verification | scrypt N=131072, r=8, p=1 |
| Evidence | 15 cases × 5 modes × 2 runs |

Original experiment record. Fresh browser captures: 1 November 2026.

# The vulnerable session decision

**CLIENT VALUES ENTER THE QUERY**

```
const filter = {
  username: req.body.username,
  password: req.body.password
};
const account = await users.findOne(filter);
```

---

```
if (account) createSession(account);
```

Condensed vulnerable excerpt. A match is incorrectly accepted as password proof.

# The hardened session decision

**1** Validate exactly two non-empty strings

```
if (!validContract(body)) return reject(400);
```

**2** Look up by username only

```
const account = await users.findOne({ username });
```

**3** Verify a salted password hash

```
const ok = await verifyPassword(password, storedHash);
```

**4** Create a session after verification

```
if (account && ok) createSession(account);
```

Condensed code. Full source includes field/length checks and a dummy hash for unknown users.

# Case results C01 to C08

| Case | Request shape | Three vulnerable modes | Protected modes |
|---|---|---|---|
| C01 | Valid credentials | Login succeeds | Login succeeds |
| C02 | Wrong string password | Login rejected | Login rejected |
| C03 | Unknown username | Login rejected | Login rejected |
| C04 | Password operator object | Attack succeeds | Stopped before query |
| C05 | Username operator with wrong password | Login rejected | Stopped before query |
| C06 | Operator-looking literal string | Login rejected | Login rejected |
| C07 | Null password | Login rejected | Stopped before query |
| C08 | Array password | Login rejected | Stopped before query |

Vulnerable modes: baseline, scripting disabled and read-only. Both runs agree.

HTTP codes are secondary evidence: 200 accepted, 401 login rejected, 400 stopped before query.

# Case results C09 to C15

| Case | Request shape | Three vulnerable modes | Protected modes |
|---|---|---|---|
| C09 | Missing password | Login rejected | Stopped before query |
| C10 | Numeric password | Login rejected | Stopped before query |
| C11 | Extra field with valid credentials | Request accepted | Stopped before query |
| C12 | Top-level array | Login rejected | Stopped before query |
| C13 | Both fields contain operators | Attack succeeds | Stopped before query |
| C14 | Empty password | Login rejected | Stopped before query |
| C15 | Empty username | Login rejected | Stopped before query |

Vulnerable modes: baseline, scripting disabled and read-only. Both runs agree.

HTTP codes are secondary evidence: 200 accepted, 401 login rejected, 400 stopped before query.

# The controls were independently checked

| Control | Independent observation |
| --- | --- |
| Read-only identity | Authenticated as labreader with the read role |
| Read permitted | Alice lookup succeeds |
| Write denied | MongoDB blocks the write (error 13) |
| Scripting enabled | The fixed $where probe returns Alice |
| Scripting disabled | The same probe fails as expected (error 6108304) |

**Both controls are active, but neither stops attack C04.**

The fixed $where expression tests configuration, not attacker-controlled JavaScript.

# Captured evidence: attack succeeds

**VULNERABLE LOGIN**

```
{"username":"alice","password":{"$ne":""}}
```

**SAME ATTACK REQUEST AND ACTUAL RESPONSE**

**HTTP 200: Session created**

**2. Database predicate**

```
{
  "username": "alice",
  "password": {
    "$ne": ""
  }
}
```

Database query: issued

**3. Application decision**

```
{
  "authenticated": true,
  "account": "alice"
}
```

**Alice: private synthetic project notes**

# Captured evidence: attack stopped

**HARDENED LOGIN**

```
{"username":"alice","password":{"$ne":""}}
```

**SAME ATTACK REQUEST AND ACTUAL RESPONSE**

**HTTP 400: Credential contract rejected**

**2. Database predicate**

```
Rejected before filter construction
```

**3. Application decision**

```
{
  "authenticated": false,
  "account": null
}
```

Database query: none

**Protected notes: access denied**

# What remains untested

**ESTABLISHED**

This operator attack succeeds
Behaviour of five configurations
Rejection before a find query
Preservation of correct login

**NOT ESTABLISHED**

A JavaScript exploit
Real-world prevalence
All drivers or DB versions
Administrator access
Complete production security

**Next experiment: permitted search without cross-tenant access.**

# Paper and database references

Ron, Shulman-Peleg and Puzanov (2016)
Analysis and Mitigation of NoSQL Injections
IEEE Security & Privacy 14(2), 30–39

DOI: 10.1109/MSP.2016.36

Author-uploaded manuscript and authors' InfoQ republication
Original classes: tautologies, union queries, JavaScript injection, piggybacked queries and origin violation.

MongoDB 7.0: $ne and $where documentation
Full URLs in the notes and team guide.

# Defences and reproducible evidence

OWASP NoSQL Security Cheat Sheet
OWASP Password Storage Cheat Sheet

**github.com/Qingyu255/CS440-Project**

Evidence commit: c8252d2
Source and results are fixed at this revision.



results.json / results.csv / server.mjs / experiment.mjs
Fresh captures and team guide accompany the revised deck.